MDPI

Article

Open or Frontier? A Cost- and Energy-Aware Benchmark of Large Language Models for Software Vulnerability Detection

Patrick Deininger [1,2,*], and Wolfgang Slany [1]

[1] Institute of Software Engineering and Artificial Intelligence, Graz University of Technology, 8010 Graz, Austria; patrick.deininger@student.tugraz.at (P.D.); wolfgang.slany@tugraz.at (W.S.)
[2] Institute of Computer Science and Artificial Intelligence, FH JOANNEUM – University of Applied Sciences, 8020 Graz, Austria
* Correspondence: patrick.deininger@student.tugraz.at

Abstract

(1) Background: Large language models (LLMs) are increasingly applied to software vulnerability detection. Yet evaluations overwhelmingly report accuracy while ignoring the monetary and environmental cost of inference, and they under-represent openly available (open-weight) models relative to proprietary frontier systems. (2) Methods: We benchmark eight contemporary LLMs, three frontier (Claude Sonnet 5, GPT-5.1, Gemini 3.1 Pro) and five open-weight (DeepSeek-V3.2, GLM-5, Llama-3.3-70B, Qwen3-Coder-30B, Gemma-3-4B), on a stratified 1549-function subset of the label-clean PrimeVul dataset. We treat cost per task and energy as first-class evaluation axes alongside detection quality, and report latency as a secondary, load-confounded observation. Monetary cost and latency are measured directly. Energy is estimated with a transparent FLOP-based model for the API-served models and measured directly on a controlled GPU for the two locally servable open models. (3) Results: The central finding concerns efficiency. Open-weight models occupy the quality and efficiency Pareto frontier, and no frontier model is Pareto-optimal. A 4-billion-parameter open model matches or exceeds frontier detection quality at roughly one-hundredth the monetary cost (at current API pricing) and one to two orders of magnitude less energy per task. This separation holds across configurations and widens when the frontier models are allowed to reason. On quality we adopt balanced accuracy as the primary metric, because no LLM beats a trivial always-safe baseline on raw accuracy. The three highest-scoring models are all open-weight (DeepSeek-V3.2 0.676, Gemma-3-4B 0.653, GLM-5 0.637, against 0.616 to 0.623 for the frontier models), and on a Holm-corrected paired bootstrap DeepSeek-V3.2 significantly exceeds all three frontier models in a compute-matched direct configuration. Allowing the frontier to reason erases this quality gap, and its best model reaches the best open model, but it does so at four to 7.5 times the cost, which leaves the efficiency conclusion untouched. Absolute quality is modest throughout. At realistic prevalence every model yields low precision, a classical static analyzer is competitive, and direct on-GPU measurement of two open models confirms the FLOP energy estimate to within roughly a factor of two. (4) Conclusions: For this task the efficiency-optimal choice, a small open-weight model, is also quality-competitive in every configuration. This has direct implications for on-premises security tooling and its energy and carbon footprint.

Keywords: large language models; vulnerability detection; software security; benchmarking; energy efficiency; open-weight models; inference cost; green AI

Received:
Accepted:
Published:

1. Introduction

Large language models (LLMs) have moved quickly from code completion into security analysis, and software vulnerability detection is a prominent target. The promise is an assistant that reads a function and flags whether it contains an exploitable weakness. A growing body of work evaluates LLMs on this task [1–3], and dedicated benchmarks now exist across both defensive detection and offensive capability assessment [4]. Two blind spots persist, however, and together they motivate this study.

First, evaluations are almost exclusively accuracy-centric. In deployment, an organization that runs vulnerability triage over a large codebase cares not only whether a model is correct but what each verdict costs in money and energy, and how long it takes. The general LLM community has begun to treat efficiency as a first-class evaluation axis [11,14,15], but this machinery has not been imported into security-task benchmarking, where the environmental and economic footprint of inference remains essentially unreported [4].

Second, security evaluations lean heavily on proprietary frontier models. A recent review of offensive-security LLM studies found that proprietary systems appeared in nearly every publication while open-weight models were used in roughly half [4]. Yet open-weight models are precisely what many security teams need. They allow on-premises deployment for privacy, compliance, and data-residency reasons, without sending sensitive source code to an external service [5].

This paper brings the two concerns together. We benchmark eight contemporary LLMs, a spread of open-weight and frontier systems, on realistic vulnerability detection, reporting cost and energy as first-class axes, with latency as a confounded secondary measure. We also calibrate the LLM results against non-LLM reference points, a static analyzer and a learned detector, to gauge absolute difficulty. Our contributions are:

- A cost- and energy-aware benchmark of open-weight versus frontier LLMs on the label-clean PrimeVul dataset [1], with a reproducible, test-driven measurement harness and non-LLM reference points (Section 3).
- Empirical evidence that open-weight models dominate the quality and efficiency Pareto frontier, from a 4-billion-parameter model at the efficiency end to a sparse mixture-of-experts at the quality end. They deliver comparable or better detection quality at one to two orders of magnitude lower cost and energy per task, and this efficiency dominance holds across configurations. It in fact widens even when the frontier models are allowed their native reasoning mode (Section 4). On quality alone, the best open model significantly beats every frontier model in a compute-matched direct configuration, and the frontier reaches parity only by reasoning at several times the cost.
- A precise account of what can and cannot be measured for closed API models. We measure cost and latency directly throughout, measure energy on-GPU for the two locally servable open models, and give a bounded, assumption-explicit FLOP estimate of energy for the API-only models. The measured points corroborate that estimate to within roughly a factor of two (Section 3).

We are explicit that these results are a first, deployment-realistic snapshot. Energy is measured directly for the two locally servable open models and FLOP-estimated for the API-only models, and the models are evaluated in a direct-answer configuration. Section 5 details these limitations and the extensions they invite.

https://doi.org/10.3390/computers1010000

2. Related Work

2.1. LLMs for Vulnerability Detection

Benchmarks for LLM-based vulnerability detection have proliferated. Meta's CyberSecEval suite evaluates insecure-code generation and broader cyber-safety [3]. SecVulEval provides a large, de-duplicated, CVE-grounded C/C++ set with statement-level labels [2]. Independent evaluations temper early optimism. Ullah et al. find that leading LLMs cannot yet reliably identify or reason about vulnerabilities [16], and Steenhoek et al. report function-level accuracy close to random [17], a picture our results corroborate. A recurring and central concern is label quality. Many widely used datasets carry substantial label noise and cross-split duplication, which inflates reported performance and collapses on realistic data [8,18]. PrimeVul was constructed specifically to address this, using near-human-accuracy labelers, rigorous de-duplication, and chronological splits, and it reports that strong code models drop from high F1 on noisy datasets to near-random on PrimeVul [1]. We adopt PrimeVul as our backbone for exactly this reason.

2.2. Efficiency and Energy of LLM Inference

Measuring the cost of inference is an active area. Direct energy measurement on datacenter GPUs has been reported for open models [9,10], and standardized power-measurement methodology exists at the systems level [11]. A well-documented pitfall is that NVIDIA's sampled power interface substantially under-samples short kernels, which motivates counter-based measurement [12]. For closed API models, energy cannot be measured directly. Infrastructure-aware estimation frameworks instead bound it from public performance data and hardware assumptions [13]. Efficiency has also entered general LLM benchmarks as a first-class axis [14,15]. Our methodology reuses this rigor but applies it, for the first time to our knowledge, within a security-detection benchmark.

2.3. The Gap

Very recent work has begun to compare open and frontier models on security detection tasks, for example web vulnerability detection in WordPress plugins [6] and dual-mode function- and application-level detection [7]. This work centers on detection accuracy, however, giving monetary cost and latency at most partial treatment and energy none. None reports energy as a co-equal per-task axis across a broad off-the-shelf open-weight roster. That combination of open-weight breadth, a security detection task, and cost and energy measured together is the gap this paper addresses.

3. Materials and Methods

3.1. Task and Dataset

We use binary function-level vulnerability detection. Given a C/C++ function, the model answers whether it contains a vulnerability. We draw from the PrimeVul test split [1] (24,788 functions after loading, of which 549 are vulnerable, a realistic 1:44 class ratio). Because the test split contains only 549 vulnerable functions, a label-stratified sample balances at most to 1098. We therefore evaluate on a stratified sample of **N=1549** (all 549 vulnerable functions plus 1000 safe functions), fixed by seed for reproducibility. Function inputs are truncated to a fixed character budget to bound cost. Each model is prompted to answer with a verdict first, followed by an optional explanation. A lenient parser extracts the verdict, and outputs that yield no parseable verdict are recorded as parse failures rather than silently counted as "safe."

https://doi.org/10.3390/computers1010000

3.2. Models

We evaluate eight models spanning parameter scale and provenance (Table 1). All are served through a unified OpenAI-compatible API gateway. Frontier models are proprietary, while open-weight models are publicly available and, in principle, self-hostable. Reasoning-capable models are run in a direct-answer configuration where the provider permits it. Gemini 3.1 Pro exposes no non-reasoning mode and is therefore run with reasoning enabled.

Table 1. Evaluated models. Active-parameter counts for frontier models are bounded assumptions used only for the energy estimate.

Model	Tier	Est. active params
Claude Sonnet 5	frontier	∼100B (assumed)
GPT-5.1	frontier	∼100B (assumed)
Gemini 3.1 Pro	frontier	∼100B (assumed)
DeepSeek-V3.2	open	∼37B active
GLM-5	open	∼32B active
Llama-3.3-70B	open	70B
Qwen3-Coder-30B	open	3B active (MoE)
Gemma-3-4B	open	4B

3.3. Measurement Methodology

We treat detection quality, cost, and energy as co-equal evaluation axes, and also report latency. Because the open-weight models here are also accessed through the API gateway, cost and latency are measured identically across all models, giving a like-for-like comparison. One caveat matters for interpretation. The cost figure is the gateway's list price, so it reflects current market pricing (which for open-weight models may be set competitively, at or below serving cost) rather than the physical cost of inference. Energy, by contrast, is the physically grounded efficiency axis. It is FLOP-estimated for the API-served models and, for the two open models we could serve locally on a single GPU (Qwen3-Coder-30B, Llama-3.3-70B), measured directly on that GPU.

Cost is computed exactly from logged input/output token counts and published per-token pricing. **Latency** is measured client-side (total wall-clock time and output tokens per second). We report it as service latency and note that the figures were collected under concurrent load and therefore conflate model speed with queueing. **Energy** cannot be measured for API-served models, which run on unknown hardware with unknown batching [13]. We therefore report a transparent FLOP-based estimate, $E \approx 2\, N_{\text{active}}\, T\, \varepsilon$, where N_{active} is the active parameter count, T the total token count, and ε a system-level energy-per-FLOP constant calibrated so that a frontier-class query reproduces published per-query estimates [13]. For open-weight models the active parameter count is known, which gives a grounded estimate. For frontier models it is a bounded assumption, and we treat the resulting energy as an estimate with an explicit uncertainty rather than a measurement. Every result row carries an `energy_source` tag so that measured and estimated energy are never conflated. For the two locally servable open models we *measure* energy directly on a dedicated NVIDIA H200 GPU, reading the NVIDIA Management Library (NVML) cumulative energy counter [10,12] across each request at concurrency 1. We report both a gross energy and an active energy that subtracts the measured idle draw. Qwen3-Coder-30B was served in `bfloat16` and Llama-3.3-70B in FP8, because the 70B model does not fit a single H200 in `bfloat16`. The remaining open models could not be served under our fixed local stack. Gemma-3-4B is incompatible with the pinned vLLM/CUDA build, and DeepSeek-V3.2 and GLM-5 exceed single-GPU capacity, so their energy remains FLOP-estimated.

3.4. Harness and Reproducibility

All experiments run through a purpose-built, test-driven harness (82 automated tests) with pluggable model backends and measurement meters. Each run records its resolved configuration, random seed, model versions, and a per-token price snapshot, and results are written incrementally so that long runs are crash-safe and resumable. Generation is deterministic (temperature 0, fixed maximum output length). As non-LLM reference points we additionally run, over the identical functions, Flawfinder 2.0.20 (a lexical C/C++ static analyzer, labeling a function vulnerable when it raises at least one hit at risk level ≥ 1) and an off-the-shelf learned detector (a CodeBERT classifier fine-tuned on the Devign dataset [20], using its argmax prediction over inputs truncated to 512 tokens). The harness, configurations, and the exact sampled function identifiers are released with this paper (see Data Availability).

3.5. Use of Generative AI

In accordance with venue policy we disclose that generative AI tools were used substantially in this work: to assist with the research design, to implement the measurement harness, to run the experiments, and to draft this manuscript. All AI-generated code is covered by the automated test suite, all quantitative results are produced by the released harness from logged data, and the authors have reviewed and take responsibility for the content. No AI system was used to fabricate data or citations.

4. Results

4.1. Choice of Primary Metric

Our sample is imbalanced (549 vulnerable, 1000 safe), so raw accuracy is dominated by a trivial classifier. Predicting "safe" for every function yields 0.646 accuracy, which exceeds the accuracy of *every* LLM we tested. Raw accuracy is therefore not a meaningful primary metric here. We instead adopt **balanced accuracy** (the mean of true-positive and true-negative rates) as the primary metric, for which both trivial classifiers score exactly 0.5, and we report the Matthews correlation coefficient (MCC) and F1 (the harmonic mean of precision and recall) alongside it. Metrics are computed over each model's parsed predictions, so the effective sample size ranges from 1526 to 1549 across models. Parse rates were near-perfect (0.985–1.000) once reasoning-capable models were configured to emit a verdict rather than only internal reasoning.

4.2. Detection Quality

Table 2 reports all eight models plus the two trivial baselines, sorted by balanced accuracy. All comparisons here use the compute-matched direct-answer mode. Section 4.5 examines what changes when the frontier models are allowed to reason. Three findings hold. First, the three highest-quality models are all open-weight. DeepSeek-V3.2 (0.676, 95% CI $[0.654, 0.697]$), Gemma-3-4B (0.653, $[0.630, 0.676]$), and GLM-5 (0.637, $[0.619, 0.654]$) all rank above the three frontier models (0.616–0.623). Because balanced accuracy, not raw accuracy, is our primary metric, we test the pairwise gaps with a stratified paired bootstrap of the balanced-accuracy difference (20,000 resamples, resampling the 549 positive and 1000 negative task outcomes independently so the design is preserved) and control the family-wise error rate across the nine comparisons between these three open models and the three frontier models with a Holm correction. DeepSeek-V3.2 significantly exceeds all three frontier models (balanced-accuracy gaps $+0.054$ to $+0.060$, Holm-adjusted $p = 7 \times 10^{-4}$ against Gemini-3.1-Pro and below the bootstrap's 10^{-4} resolution against the other two). These conclusions do not depend on the choice of family. Under a stricter Holm correction over all fifteen open-versus-frontier pairs, DeepSeek still beats all three, and Gemma-

3-4B's edge over Claude-Sonnet-5 remains significant (adjusted $p = 0.049$). The next two open models rank above every frontier model on the point estimate, but by smaller margins that mostly do not survive correction. After Holm adjustment, only Gemma-3-4B's advantage over Claude-Sonnet-5 remains significant ($+0.037$, $p = 0.03$). Its gaps to GPT-5.1 ($+0.036$, $p = 0.08$) and Gemini-3.1-Pro ($+0.028$, $p = 0.13$), and all three of GLM-5's gaps ($+0.012$ to $+0.021$, $p \geq 0.13$), do not. The robust claim is therefore narrower than a blanket one. The single best open model significantly beats the entire frontier tier, and the three best open models all rank above it, but not every top-open-versus-frontier gap is individually significant, and not every open model beats the frontier. Qwen3-Coder-30B is indistinguishable from chance (balanced accuracy 0.511, MCC 0.022, 95% CI on MCC $[-0.03, 0.07]$). On the threshold-invariant MCC the separation is narrower still. DeepSeek-V3.2 (0.344) leads clearly, but Gemini-3.1-Pro (0.298) is level with Gemma-3-4B (0.296) and above GPT-5.1 (0.230), so the unambiguous quality result is DeepSeek's rather than a blanket open-weight advantage.

Second, absolute quality is modest for all models (balanced accuracy 0.51–0.68, MCC 0.02–0.34), and specificity varies widely, from 0.25 (Llama-3.3-70B) to 0.57 (Qwen3-Coder-30B). This matters for deployment. On PrimeVul's natural 1:44 prevalence, rather than on our balanced sample, precision is low for every model, at 2.3–3.8%. That is only marginally above the 2.2% base rate an always-flag classifier would achieve. The balanced-accuracy ranking thus identifies the *least miscalibrated* model, not a deployment-ready one, and we return to this in Section 5. Third, the frontier models in particular show high recall but low specificity (Gemini-3.1-Pro and Claude-Sonnet-5 flag about 70% of safe functions), which is why they trail on the balanced metric despite competitive F1.

Table 2. Detection quality and efficiency on PrimeVul (N=1549, direct-answer mode), sorted by balanced accuracy. Balanced-accuracy 95% CIs and Holm-corrected paired significance tests are given in Section 4.2. **Spec.** is specificity (true-negative rate). Cost is measured for all models. Energy is measured on a dedicated H200 GPU for the two locally served open models (†, active energy, see Section 4.4) and FLOP-estimated otherwise. Flawfinder is a lexical static-analysis reference point (any hit at risk level $\geq$1). CodeBERT-Devign is an off-the-shelf CodeBERT classifier fine-tuned on the Devign dataset, applied cross-dataset. Both are local, with negligible cost and energy. Best per column in bold, except accuracy (deprecated here), the energy column (mixed measured and estimated), and specificity.

Model	Tier	Bal. acc.	MCC	F1	Acc.	Recall	Spec.	USD/task	Energy (J)
DeepSeek-V3.2	open	**0.676**	**0.344**	**0.615**	0.629	0.836	0.515	0.00017	526
Gemma-3-4B	open	0.653	0.296	0.590	0.617	0.778	0.528	**0.00004**	64
GLM-5	open	0.637	0.307	0.596	0.548	0.940	0.334	0.00048	424
Gemini-3.1-Pro	frontier	0.623	0.298	0.591	0.526	**0.963**	0.284	0.00448	1966
GPT-5.1	frontier	0.617	0.230	0.560	0.573	0.769	0.465	0.00139	1332
Claude-Sonnet-5	frontier	0.616	0.272	0.582	0.522	0.940	0.292	0.00441	2489
Llama-3.3-70B	open	0.604	0.260	0.578	0.503	0.956	0.252	0.00008	738†
Qwen3-Coder-30B	open	0.511	0.022	0.405	0.529	0.452	0.571	0.00006	88†
Flawfinder (static analysis)	–	0.589	0.235	0.377	0.682	0.271	0.907	≈0	≈0
CodeBERT-Devign (learned)	–	0.518	0.105	0.532	0.378	0.996	0.039	≈0	≈0
Always-safe (baseline)	–	0.500	0.000	0.000	0.646	0.000	1.000	–	–
Always-vulnerable (baseline)	–	0.500	0.000	0.523	0.354	1.000	0.000	–	–

4.3. Efficiency and the Pareto Frontier

The efficiency separation is order-of-magnitude and robust to the choice of quality metric. Gemma-3-4B exceeds Claude-Sonnet-5 on balanced accuracy (0.653 against 0.616) while costing roughly one-hundredth as much per task ($0.00004 against $0.0044) and consuming one to two orders of magnitude less energy. The estimated 64 J against 2489 J is a 39× gap at our assumed frontier active-parameter count, and it spans roughly 10–150×

across the plausible range of that assumption (Section 4.4). Service latency, measured client-side but under concurrent load, and therefore conflating model speed with queueing (see Section 5.3), points the same way. Gemma-3-4B is the fastest model at 1.4 s mean wall-clock time per task, against 3.4–4.7 s for the slowest four (DeepSeek-V3.2, GLM-5, Gemini-3.1-Pro, Claude-Sonnet-5). Figures 1 and 2 plot balanced accuracy against cost and energy on log axes. The small open-weight models occupy the upper-left corner, with high quality at low cost and energy, and no frontier model is Pareto-optimal on either axis.

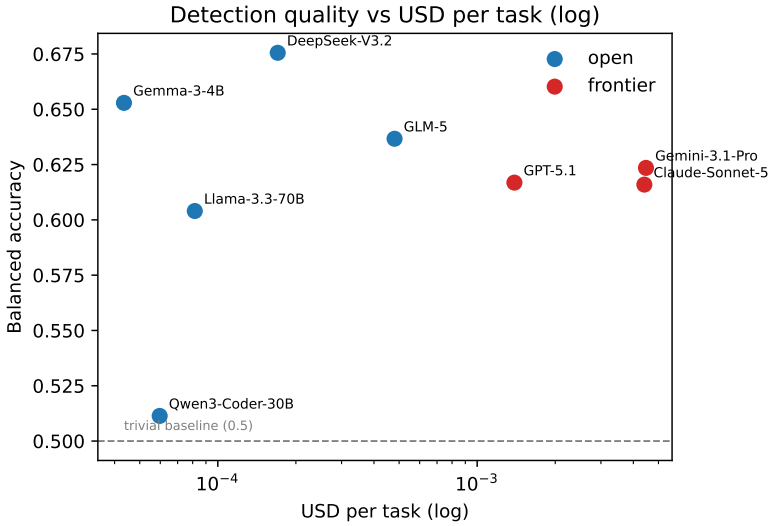

Figure 1. Balanced accuracy versus measured monetary cost per task (log scale). Open-weight models (blue) dominate the frontier models (red). The dashed line marks the trivial-baseline balanced accuracy of 0.5.

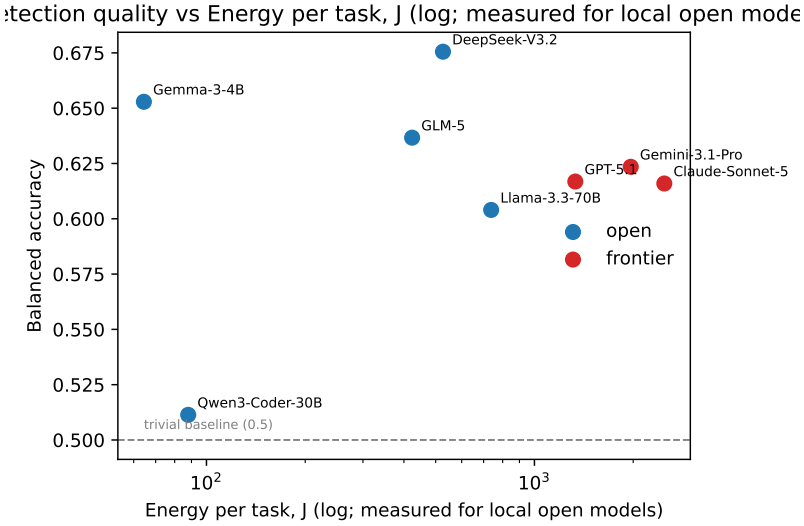

Figure 2. Balanced accuracy versus energy per task (log scale). Energy is measured on an H200 GPU for Qwen3-Coder-30B and Llama-3.3-70B (Section 4.4) and FLOP-estimated for the other six models. Substituting the measured values for the prior estimates leaves the ordering unchanged.

4.4. Measured Versus Estimated Energy

For the two open models we could serve locally, we replace the FLOP estimate with direct H200 measurement and ask how far the estimate was off (Table 3).[1] The estimate agrees with the measurement to roughly a factor of two (0.8× to 2.2×), and it errs in opposite directions by architecture. For the sparse Qwen3-Coder-30B (3B active of 30B total) the estimate *undershoots* the measurement by 2.2× (40 against 88 J). An active-parameter FLOP count captures neither the memory traffic of the full expert set nor the low GPU utilization of single-request decode. For the dense Llama-3.3-70B, served in FP8, the estimate *overshoots*. At 944 J it is about 1.3 times the 738 J measurement (equivalently, the measurement is 0.8× the estimate, as in Table 3), because the estimate is precision-agnostic while the measurement benefits from 8-bit arithmetic. Measured energy per output token, 1.4 J for the 3B-active MoE and 11.6 J for the 70B model, *straddles* the roughly 3–4 J/token reported for datacenter GPUs by Samsi et al. [10]. Neither value lands inside that band, but they bracket it and scale with model size as expected, which is the most this loose cross-hardware check can establish.

Two cautions bound what this validates. It confirms the estimator's structure and its energy-per-FLOP constant for small and mid-size *open* models under single-request serving. It does not test the assumed frontier active-parameter count (Table 1), which is the dominant uncertainty in the frontier energy figures and cannot be measured here. The measurement is also taken at concurrency 1, a low-utilization and energy-pessimistic regime, while the estimator's constant is calibrated to batched datacenter serving, so exact agreement is not expected. The robust and measurement-independent conclusion is narrower, and it is the one we rely on. The measured values move both open models only within their order of magnitude. They keep both models well below every frontier estimate and leave the Pareto frontier and the open-versus-frontier separation unchanged (Figure 2). The only reordering is at the extreme low-energy end, where Gemma-3-4B (estimated 64 J) rather than Qwen3-Coder-30B (measured 88 J) is now the least energy-intensive model. The efficiency gap is so large that even a two-fold estimation error, in either direction, does not threaten it.

Table 3. Measured (H200, NVML) versus FLOP-estimated energy per task for the two locally served open models. Active energy subtracts the measured idle draw. The ratio is measured-active over estimated.

Model	Precision	Estimated (J)	Measured active (J)	Meas./est.	J / out. tok
Qwen3-Coder-30B	bfloat16	40	88	2.2×	1.4
Llama-3.3-70B	FP8	944	738	0.8×	11.6

4.5. Reasoning versus Direct Answering

Our main comparison uses direct-answer mode so that all models are judged at a comparable single-shot budget. Because reasoning is the frontier models' native mode, we test its effect at full scale. We re-run all three frontier models with reasoning enabled on the full N=1549, and pair Claude-Sonnet-5 and GPT-5.1 against their direct-mode predictions. Reasoning raises balanced accuracy by +0.030 (Claude, from 0.616 to 0.646, paired bootstrap $p = 0.013$) and +0.043 (GPT-5.1, from 0.617 to 0.660, $p = 0.002$), and both gains are significant. Gemini-3.1-Pro, which has no direct mode and was capped at 256

[1] The measurement is a dedicated $N = 300$ run at concurrency 1. The "Estimated" column is the mean FLOP estimate from the main $N = 1549$ run. Mean output length matched across the two (about 64 tokens), so the per-task comparison is not materially confounded by the differing subset, though the two are not the identical requests.

output tokens in the main run, rises to 0.677 given a 4096-token budget, up from 0.623, which shows its main-run score was budget-limited. These gains come at roughly 18× the generated tokens (mean 1137 against the 64-token direct budget) and several times higher cost per task, up to 7.5× for GPT-5.1.

To make the comparison symmetric, we also ran the two open models with native reasoning modes, DeepSeek-V3.2 and GLM-5, with reasoning enabled. Reasoning helps neither. For DeepSeek-V3.2 it significantly *hurts*, dropping balanced accuracy from 0.676 to 0.608 (paired difference -0.067, $p < 0.001$) at roughly thirty times the output tokens. For GLM-5 it is slightly negative and not significant (0.637 to 0.621, -0.017, $p = 0.08$). The cause is calibration. Reasoning lowers DeepSeek's specificity from 0.52 to 0.37, so a model that was already well-balanced begins to over-flag. The best open model is therefore best in its direct configuration, and reasoning is not a free improvement but one that helps the frontier (whose direct scores were low) while doing nothing for, or degrading, an already well-calibrated open model.

Two conclusions follow. First, reasoning brings the frontier up to parity at the very top, but no further. The strongest frontier-with-reasoning model (Gemini-3.1-Pro, 0.677) matches the best open model (DeepSeek-V3.2, 0.676 in direct mode), while Claude and GPT-5.1 improve significantly but remain below it. Because reasoning helps the frontier yet hurts DeepSeek, no configuration of any model, open or frontier, exceeds DeepSeek's direct-mode 0.676. The open models' quality *lead* is thus specific to the compute-matched direct configuration, and reasoning closes it to a tie, but the frontier does not surpass the best open model in any configuration we tested. Second, because reasoning multiplies cost and energy several times over, it *widens* the efficiency gap, so the open models' efficiency dominance is robust to, and reinforced by, allowing the frontier to reason.

5. Discussion

5.1. Open-Weight Models on the Efficiency Frontier

The central result is that open-weight models occupy the quality and efficiency Pareto frontier, and no frontier model is Pareto-optimal. That frontier is spanned by two open models: the 4-billion-parameter Gemma-3-4B at the low-cost, low-energy end, and the sparse mixture-of-experts DeepSeek-V3.2 at the high-quality end. Allowing the frontier its native reasoning mode widens the efficiency gap while only bringing frontier quality up to parity with the best open model (Section 4.5). On balanced accuracy per dollar and per Joule, small open models such as Gemma-3-4B beat every frontier system by one to two orders of magnitude (Figures 1, 2). Neither efficiency axis is assumption-free. The cost figure is measured but is API list price, a market quantity that for open models may sit below serving cost, while the energy figure is physically grounded but partly estimated. They point the same way regardless, and both separations are one to two orders of magnitude, far larger than the uncertainty in either. This comes with an essential qualification. As Section 4.2 shows, every model is near chance in absolute terms, and at PrimeVul's realistic 1:44 prevalence even the best model raises roughly twenty-six false positives per true positive. The finding is therefore conditional. If an organization deploys an LLM for function-level triage, the open models are both the higher-quality choice (DeepSeek-V3.2) and the far cheaper one (down to the 4-billion-parameter Gemma-3-4B), but this is not an endorsement of the task as solved. We also flag that our cost figures are the API gateway's prices for every model. A genuine on-premises deployment substitutes amortized hardware, electricity, and utilization for that price, so the break-even volume above which self-hosting actually wins is a modeling question we leave to future work.

Translating energy to carbon makes the environmental stake concrete while keeping it bounded. At a representative grid intensity of 250 gCO_2eq/kWh and a datacenter PUE

of 1.2, Gemma-3-4B's 64 J per task is about 5 mgCO_2eq, against roughly 210 mgCO_2eq for Claude-Sonnet-5. Over a million-function codebase that is about 5 kg against about 210 kg, and about 10 against about 390 kg at the world-average 475 gCO_2eq/kWh. These are operational GPU-energy figures only. They exclude embodied hardware carbon, which at low on-premises utilization would count against the smaller-model case, so we treat them as illustrative rather than as a life-cycle assessment. They are also assumption-bearing. Claude-Sonnet-5's energy is a FLOP estimate resting on an assumed active-parameter count, so the frontier carbon figure and the roughly 40-fold ratio inherit that assumption's 10 to 150-fold sensitivity, and the ratio compares one estimate against another, since Gemma-3-4B's energy is estimated too. The absolute footprints are modest in isolation, but the *ratio* is large and recurring, and it compounds with every re-scan of an evolving codebase (though a cheaper per-task scan may also induce more scanning, a rebound effect that bounds the aggregate saving). This is a concrete instance of the "green AI" argument [9,14] applied to a security workload. The result is not that open models are uniformly superior, since Qwen3-Coder-30B is indistinguishable from chance, but that the best open models dominate the frontier on both the quality and the efficiency axes.

5.2. Why the Benchmark Is Hard

No LLM exceeded 0.68 balanced accuracy, and none beat the trivial always-safe classifier on raw accuracy, an important sanity check on this imbalanced set. Two non-LLM reference points frame the difficulty (Table 2). A classical lexical static analyzer, Flawfinder, reaches 0.589 balanced accuracy, below every LLM except the near-chance Qwen3-Coder-30B. Because it flags conservatively (specificity 0.91), it is the only method here that beats the always-safe baseline on raw accuracy (0.682 against 0.646) and the most precise at realistic prevalence (6%), at essentially zero cost. A learned detector, CodeBERT fine-tuned on the Devign dataset and applied off the shelf, does *worse*. It collapses to 0.518 balanced accuracy, flagging 97% of functions as vulnerable, which is essentially the always-vulnerable baseline. This is precisely PrimeVul's thesis, that a model trained on an older, noisier dataset does not transfer to its clean, de-duplicated test set [1,8,18]. On balanced accuracy the best LLMs (DeepSeek-V3.2, 0.676) thus outperform both a classical static analyzer and a learned detector, and they are the strongest tools available here. The picture flips on precision at deployment prevalence, though, where the conservative Flawfinder (6%) is ahead of every LLM (2.3 to 3.8%). All methods remain far from reliable, which is exactly what makes the benchmark meaningful. Several models default to flagging. Gemini-3.1-Pro and Claude-Sonnet-5 among the frontier systems, and GLM-5 and Llama-3.3-70B among the open ones, all have specificity below 0.36, which in a triage setting translates into alert fatigue rather than useful signal. DeepSeek-V3.2 and Gemma-3-4B lead the balanced metric because they pair competitive recall with specificity above 0.5. Qwen3-Coder-30B is also conservative (specificity 0.57) but couples it with a recall of only 0.45, so its balanced accuracy is barely above chance and it gains nothing.

5.3. Limitations and Threats to Validity

Several limitations bound our claims and shape the measured extensions we are pursuing.

- **Energy is only partly measured.** We measure energy directly on an H200 GPU for the two locally servable open models. For the three frontier models (unknown hardware) and the three open models we could not serve locally, energy remains a FLOP-based estimate. The measured points agree with that estimate to within roughly a factor of two and preserve the Pareto frontier and open-versus-frontier separation (Section 4.4), but they validate only the open-model regime. The assumed frontier active-parameter

count, which drives the frontier energy magnitudes, is untested, so we report those magnitudes as a sensitivity range rather than as point values.

- **Direct-answer configuration.** The main comparison disables reasoning where the provider allows it, for a comparable single-shot judgment. Section 4.5 quantifies the effect at full scale. Reasoning buys a significant $+0.03$ to $+0.05$ balanced accuracy at four to 7.5 times the cost, bringing the strongest frontier model to parity with the best open model but not above it. The comparison is symmetric. Enabling reasoning on the two open models with reasoning modes (DeepSeek-V3.2, GLM-5) helps neither, and it significantly lowers DeepSeek's balanced accuracy to 0.608, so the open models need no reasoning and the parity holds both ways. Our main-run 256-token cap also understated Gemini-3.1-Pro, which reaches 0.677 at a 4096-token budget. Because reasoning multiplies cost and energy, it cannot change the efficiency ordering.
- **Decoding-budget asymmetry.** The two reasoning-only frontier models (Claude-Sonnet-5, Gemini-3.1-Pro) need a larger output budget (256 against 64 tokens) to emit a verdict and are reported at that budget. Because they are already the efficiency losers, their reported cost and energy gap is conservative. GLM-5, which we initially ran at the larger budget, is re-run at the matched 64-token budget with reasoning disabled and reported there. Its balanced accuracy is essentially unchanged (0.637 against 0.647), which confirms the budget does not drive its result.
- **Latency under concurrency.** The service-latency figures we report (Section 4) were collected under concurrent load and conflate model speed with queueing. A controlled single-request latency pass is future work, so we treat latency as a directional observation rather than as a precise axis.
- **Training-data contamination.** PrimeVul is built from public CVE-fixing commits. All of them here are from 2008–2022, predating the models' training cutoffs, and they may appear in pretraining data. Binning the vulnerable functions by CVE year reveals a modest memorization gradient. Recall on pre-2020 CVEs is 7–15 points higher than on 2021–2022 CVEs for the better-balanced models (for example GPT-5.1 0.88 against 0.73, and DeepSeek-V3.2 0.90 against 0.81), consistent with some memorization of older, well-documented vulnerabilities. The gradient appears in both tiers, so it does not obviously explain the open-versus-frontier gap, but it does suggest absolute scores may be optimistic. A cleaner bound would restrict to post-cutoff CVEs or to PrimeVul's paired vulnerable and patched variants, which resist superficial memorization, and is planned.
- **Single sample, single run.** Results use one stratified draw of the 1000 safe functions, one seed, and one generation per item at temperature 0. The paired bootstrap (Section 4.2) quantifies within-sample uncertainty, but because the balanced-accuracy ranking is specificity-driven, resampling the safe pool and repeating generations would additionally bound draw-to-draw and API run-to-run variance for the closely-spaced middle of the table.
- **Baselines.** We include a lexical static-analysis baseline (Flawfinder) and an off-the-shelf learned detector (CodeBERT fine-tuned on Devign) alongside the trivial classifiers. Both are cross-tool and cross-dataset reference points. A learned detector fine-tuned directly on PrimeVul's training split (for example in the LineVul [19] family) and a semantic analyzer such as CodeQL would further calibrate the absolute difficulty, and both are natural extensions. PrimeVul's own authors report that models fine-tuned on its training split also remain near-random [1], consistent with our off-the-shelf result.
- **Scope.** We report one dataset, one task formulation, and a balanced sample of 1549 functions. Generalization to other languages, to statement-level localization, and

to the realistic-imbalance regime (via a scored metric such as VD-Score [1]) remains future work.

None of these caveats reverses the headline. The direction of both quality and efficiency favors open-weight models. Cost is measured for all models, energy is measured for two of them and agrees with the estimate to within roughly a factor of two, and the efficiency gaps are one to two orders of magnitude, far larger than any of these uncertainties.

6. Conclusions

We benchmarked eight open-weight and frontier LLMs on label-clean vulnerability detection, measuring cost, latency, and energy alongside detection quality. The robust finding is efficiency. Open-weight models dominate the quality and efficiency Pareto frontier, no frontier model is Pareto-optimal, and this holds in every configuration we tested, at roughly one-hundredth the cost and one to two orders of magnitude less energy per task. On quality, in a matched direct-answer configuration the best open model significantly exceeds every frontier model on balanced accuracy (DeepSeek-V3.2 beats all three), while all models remained close to chance on this hard benchmark. Allowing the frontier models to reason erases the quality gap, and their strongest model reaches but does not surpass the best open model, at several times the cost, so the efficiency conclusion holds across configurations. That last caveat is load-bearing. At realistic prevalence every model raises far more false alarms than true ones, so the practical claim is conditional. If an LLM is deployed for function-level triage, the open models are the accuracy-motivated and efficiency-motivated choice. Direct H200 measurement of two locally servable open models shows the FLOP estimate agrees to within roughly a factor of two and leaves the Pareto ordering intact. Extending on-GPU measurement to the remaining self-hostable models, adding a detector fine-tuned on PrimeVul, and corroborating the efficiency ordering on a second dataset are the immediate next steps.

Author Contributions: Conceptualization, P.D. and W.S.; methodology, P.D.; software, P.D.; validation, P.D.; formal analysis, P.D.; investigation, P.D.; data curation, P.D.; writing (original draft preparation), P.D.; writing (review and editing), P.D. and W.S.; visualization, P.D.; supervision, W.S.; project administration, P.D. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the State of Styria (Land Steiermark), Office of the Styrian Provincial Government, Department 12 (Economy, Tourism, Science and Research), within the PRISMA project, grant number ABT12-270413/2024.

Institutional Review Board Statement: Not applicable. This study did not involve humans or animals. It evaluates the outputs of large language models on the publicly available PrimeVul dataset.

Informed Consent Statement: Not applicable.

Data Availability Statement: The measurement harness, all run configurations, the exact sampled function identifiers, and the aggregated results are released at [repository URL]. The PrimeVul dataset is available from its original authors [1].

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

References

1. Ding, Y.; et al. Vulnerability Detection with Code Language Models: How Far Are We? In *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2025; arXiv:2403.18624.

2. Ahmed, M.B.U.; et al. SecVulEval: Benchmarking LLMs for Real-World C/C++ Vulnerability Detection. *arXiv* 2025, arXiv:2505.19828.
3. Bhatt, M.; et al. Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models. *arXiv* 2023, arXiv:2312.04724.
4. Happe, A.; et al. Benchmarking Practices in LLM-driven Offensive Security: Testbeds, Metrics, and Experiment Design. *arXiv* 2025, arXiv:2504.10112.
5. Levi, M.; et al. Toward Cybersecurity-Expert Small Language Models. *arXiv* 2025, arXiv:2510.14113.
6. Neef, S.; et al. Evaluating LLMs for Real-World Web Vulnerability Detection. *arXiv* 2026, arXiv:2606.21397.
7. Dahiya, V.; et al. Are Frontier LLMs Ready for Cybersecurity? Evidence for Vertical Foundation Models from Dual-Mode Vulnerability Benchmarks. *arXiv* 2026, arXiv:2605.23243.
8. Croft, R.; Babar, M.A.; Kholoosi, M.M. Data Quality for Software Vulnerability Datasets. In *Proceedings of ICSE*, 2023; arXiv:2301.05456.
9. Luccioni, A.S.; Jernite, Y.; Strubell, E. Power Hungry Processing: Watts Driving the Cost of AI Deployment? In *Proceedings of ACM FAccT*, 2024; arXiv:2311.16863.
10. Samsi, S.; et al. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. In *Proceedings of IEEE HPEC*, 2023; arXiv:2310.03003.
11. Tschand, A.; et al. MLPerf Power: Benchmarking the Energy Efficiency of Machine Learning Systems. *arXiv* 2024, arXiv:2410.12032.
12. Part-time Power Measurements: nvidia-smi's Lack of Attention. *arXiv* 2023, arXiv:2312.02741.
13. Jegham, N.; et al. How Hungry is AI? Benchmarking Energy, Water, and Carbon Footprint of LLM Inference. *arXiv* 2025, arXiv:2505.09598.
14. Liang, P.; et al. Holistic Evaluation of Language Models (HELM). *arXiv* 2022, arXiv:2211.09110.
15. Niu, C.; et al. TokenPowerBench: Benchmarking the Power Consumption of LLM Inference. *arXiv* 2025, arXiv:2512.03024.
16. Ullah, S.; et al. LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?): A Comprehensive Evaluation, Framework, and Benchmarks. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, 2024; arXiv:2312.12575.
17. Steenhoek, B.; et al. A Comprehensive Study of the Capabilities of Large Language Models for Vulnerability Detection. *arXiv* 2024, arXiv:2403.17218.
18. Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. Deep Learning based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering* 2022; arXiv:2009.07235.
19. Fu, M.; Tantithamthavorn, C. LineVul: A Transformer-based Line-Level Vulnerability Prediction. In *Proceedings of the IEEE/ACM 19th International Conference on Mining Software Repositories (MSR)*, 2022.
20. Zhou, Y.; Liu, S.; Siow, J.; Du, X.; Liu, Y. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019; arXiv:1909.03496.